

目 录

摘要	1
引 言	2
1.绪论	3
1.1 设计的主要内容和意义	3
1.1.1 设计防盗报警器的内容	3
1.1.2 设计防盗报警器的意义	3
2.硬件电路的设计	4
2.1 智能家居防盗报警器的硬件组成	4
2.2 智能家居防盗报警器的硬件设计	4
2.2.1 人体红外采集电路设计	4
2.2.2 光线采集电路设计	7
2.2.3 温度传感器电路设计	9
2.2.4 DS18B20 引脚介绍	10
2.2.5 DS18B20 的内部结构	11
2.2.6 烟雾传感器电路设计	12
2.2.7 MQ-2 型烟雾传感器的工作原理	12
2.2.8 MQ-2 型传感器的特性	12
2.3 AD 电路设计	13
2.3.1 ADC0809 的结构及转换原理	14
2.3.2 ADC0809 连线图	16
2.4 单片机电路设计	16
2.5 报警电路设计	20
2.6 液晶显示电路设计	20

3.软件的设计	23
3.1 KEIL 应用	23
3.2 PROTEL99SE 应用	23
3.3 DS18B20 的程序流程图	25
4.系统分析与调试	25
结 论	27
致 谢	28
参考文献	29
附录 1 源程序	30

智能家居防盗报警器

摘要

该报警器具有误报率较低、安装和配置容易、成本低、能量消耗少、使用非常方便的特点。实现的原理是温度传感器检测室内温度控制加热和降温设备工作，霍尔元件检测门的状态，热释电模块检测是否有人进入室内，光敏电阻检测室内光线，烟雾传感器检测室内是否有火灾或者可燃气体泄露。当有异常情况时，各传感器便向单片机发出信号，再经单片机处理，然后显示到液晶屏上，同时控制各负载的工作。本设计具有很强的实用价值和良好的市场前景。

关键词：报警系统，单片机，防盗系统，环境检测

引 言

智能家居防盗报警器的发展主要是基于传感器之上，所以有必要先谈谈红外传感器的发展状况。而传感器技术是 21 世纪人们在高新技术发展方面争夺的一个制高点，各发达国家都将传感器技术视为现代高新技术发展的关键。从 20 世纪 80 年代起，日本就将传感器技术列为优先发展的高新科技之首，美国等西方国家也将此技术列为国家科技和国防技术发展的重点。从而基于传感器技术的防盗报警系统也得到了高速发展。

传统的防盗报警器都是检测到有盗情的时候只在本地发出警报声音，内部没有控制器，易被破坏失效，安装，扩展也不方便。本文设计的家居防盗报警器利用单片机控制，功能强大，并且易于扩展成多用途的智能家居系统。

霍尔传感器用于检测磁性元件，当门关闭时，霍尔是有检测到信号的，当门开是，霍尔检测不到信号就会输出信号到单片机。烟雾传感器是将烟雾的浓度变化转换成电压的变化，通过 ADC0809 转换成数字量输送给单片机，单片机读取数字量，判断烟雾是否超过限值。温度传感器检测环境温度，单片机判断温度是否适合主人居住来控制加热和制冷设备工作。

1.绪论

1.1 设计的主要内容和意义

1.1.1 设计防盗报警器的内容

首先是为系统总体设计方案划分功能模块.第一部分是各传感器，第二部分是主机(含显示器,处理器,报警器等)；按键控制（对防盗器实行布防和撤防以及测试）。其次是确定硬件电路的设计，包含芯片的选择，具体电路的设计如探测电路、报警电路等等。最后就是软件的设计，软件的设计主要是以熟悉硬件电路的工作原理为前提来设计的。

1.1.2 设计防盗报警器的意义

随着我国国民经济和人民生活水平的不断提高，人们的家居生活条件不断得到改善。由于当今社会城市化进程不断加快，城市人口膨胀，外来人口的增加，城市里建成了大量现代化小区。另外，加上煤气和家用电器设备使用中的不安全因素等，对居民的生命和财产安全构成了很大的潜在威胁。经过分析，给居民生命和财产安全带来最大的威胁包括有两大方面，一方面是人为引起的破坏(如盗窃、抢劫、凶杀)，另一方面是自然灾害引起的破坏(如火灾)。因此，人们迫切需要一种对室内出现的各种紧急意外事件（如发生入室盗窃、火灾等）自动发出报警信息的智能家居系统。

传统的机械式（防盗网、防盗窗）家居防卫系统在实际使用中暴露了很多隐患，除了操作繁琐之外，还没达到安全的效果。同时，随着社会信息化进程的逐步深入，特别是传感器技术和先进的电子通信技术的迅猛发展，家居生活的网络化与智能化将成为人们新的需求，智能家居系统也由此成为了目前世界范围内的热门的研究领域之一。

本课题旨在开发出能够准确检测是否有陌生人闯入居室、是否有发生火灾的入门级家居安防报警系统。该课题的研究，符合社会的需求，与同类的智能家居系统相比具有使用简单方便、价格低廉、安装简单等优点，具有广阔的市场前景，同时有利于更好的改善人们生活水平，保障家居人身和财产安全，是一项非常具有研究和开发意义的题目。

2. 硬件电路的设计

2.1 智能家居防盗报警器的硬件组成

智能家居防盗报警器主要是由人体探测器(红外探测信号)、烟雾传感器、温度传感器、光线传感器、霍尔传感器、中央控制单元、数据显示单元、报警电路、按键控制电路、模拟输出控制和电源电路等部分组成。其框图如图 1 所示。

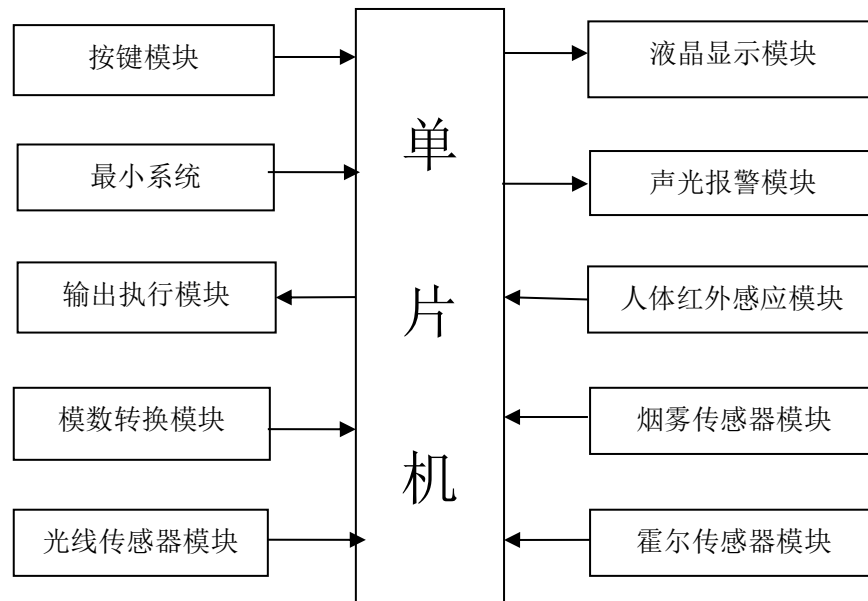


图 1 智能家居防盗报警器的组成框图

按键 1 是切换主人是否在家，只有当主人不在家时，才会开启防盗报警系统，当主人在家时，开启温度光线以及烟雾检测功能，按键 2、3 和 4 是用来设置温度和烟雾的报警值的，按键 2 是设置，按键 3 是加键，按键 4 是减键。

2.2 智能家居防盗报警器的硬件设计

2.2.1 人体红外采集电路设计

在自然界，任何高于绝对温度（-273 度）时物体都将产生红外光谱，不同温度的物体，其释放的红外能量的波长是不一样的，因此红外波长与温度的高低是相关的。

在被动红外探测器中有两个关键性的元件，一个是热释电红外传感器（PIR），它能将波长为 8—12μm 之间的红外信号变化转变为电信号，并能对自然界中的白光信号具有抑制作用，因此在被动红外探测器的警戒区内，当无人移动时，热释电红外感应器感应到的只是背景温度，当人体进入警戒区，通过菲涅尔透镜，热释电红外感应器感应到的是人体温度与背景温度的差异信号，因此，红外探测器的红外探测的基本概念就是感应移动物体与背景物体的温度的差异。

另外一个器件就是菲涅尔透镜，菲涅尔透镜有两种形式，即折射式和

反射式。菲涅尔透镜作用有两个:一是聚焦作用,即将热释的红外信号折射(反射)在 PIR 上,第二个作用是将警戒区内分为若干个明区和暗区,使进入警戒区的移动物体能以温度变化的形式在 PIR 上产生变化热释红外信号,这样 PIR 就能产生变化的电信号。

人体都有恒定的体温,一般在 37 度,所以会发出特定波长 10 微米左右的红外线,被动式红外探头就是靠探测人体发射的 10 微米左右的红外线而进行工作的。人体发射的 10 微米左右的红外线通过菲涅尔滤光片增强后聚集到红外感应源上。红外感应源通常采用热释电元件,这种元件在接收到人体红外辐射温度发生变化时就会失去电荷平衡,向外释放电荷,后续电路经检测处理后就能产生报警信号。

1) 这种探头是以探测人体辐射为目标的。所以热释电元件对波长为 10 微米左右的红外辐射必须非常敏感。

2) 为了仅仅对人体的红外辐射敏感,在它的辐射照面通常覆盖有特殊的菲涅尔滤光片,使环境的干扰受到明显的控制作用。

3) 被动红外探头,其传感器包含两个互相串联或并联的热释电元。而且制成的两个电极化方向正好相反,环境背景辐射对两个热释元件几乎具有相同的作用,使其产生释电效应相互抵消,于是探测器无信号输出。

4) 人一旦侵入探测区域内,人体红外辐射通过部分镜面聚焦,并被热释电元接收,但是两片热释电元接收到的热量不同,热释电也不同,不能抵消,经信号处理而报警。

5) 菲涅尔滤光片根据性能要求不同,具有不同的焦距(感应距离),从而产生不同的监控视场,视场越多,控制越严密。

被动式热释电红外探头的优缺点:

优点是本身不发任何类型的辐射,器件功耗很小,隐蔽性好。价格低廉。缺点是:

1. 容易受各种热源、光源干扰
2. 被动红外穿透力差,人体的红外辐射容易被遮挡,不易被探头接收。
3. 易受射频辐射的干扰。
4. 环境温度和人体温度接近时,探测和灵敏度明显下降,有时造成短时失灵。

红外线热释电传感器的安装要求:

红外线热释电人体传感器只能安装在室内,其误报率与安装位置和方式有极大的关系。正确的安装应满足下列条件:

- 1、红外线热释电传感器应离地面 2~2.2 米。
- 2、红外线热释电传感器远离空调,冰箱,火炉等空气温度变化敏感的地方。
- 3、红外线热释电传感器和被探测的人体之间不得间隔家具、大型盆景、玻璃、窗帘等其他物体。
- 4、红外线热释电传感器不能直对门窗及有阳光直射的地方,否则窗外的热气流扰动和人员走动会引起误报,有条件的最好把窗帘拉上。红外线热释电传感器也不要

安装在有强气流活动的地方。

5、安装探测器的天花板或墙要坚固，不能有晃动或震动。

红外线热释电传感器对人体的敏感程度还和人的运动方向关系很大。红外线热释电传感器对于径向移动反应最不敏感，而对于横切方向（即与半径垂直的方向）移动则最为敏感。在现场选择合适的安装位置是避免红外探头误报、求得最佳检测灵敏度极为重要的一环。

图 6 为人体感应模块内部图。本设计将人体感应模块的输出信号端，利用三极管的放大原理，将信号放大，来驱动信号给单片机。

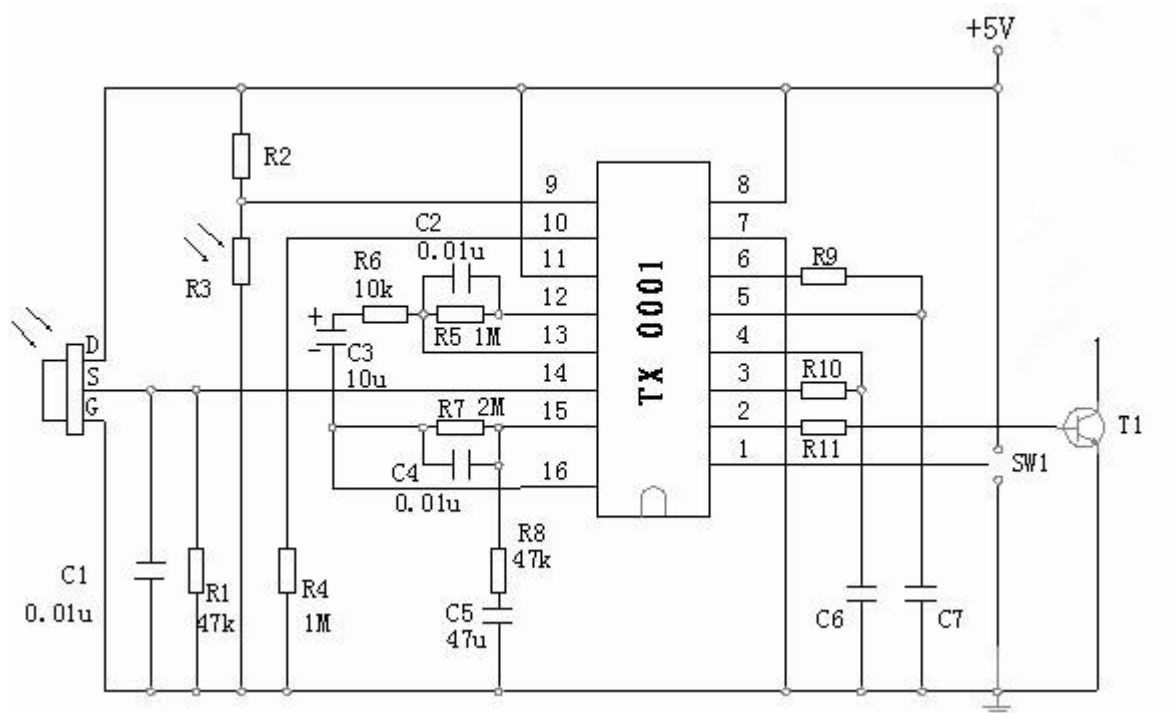
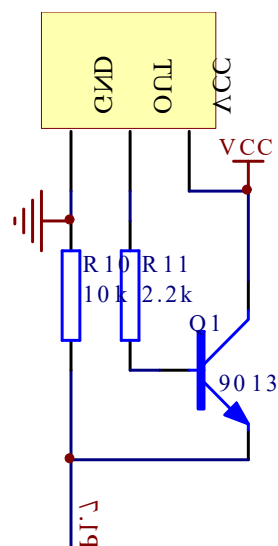


图 6 人体感应模块电路图

热释电模块连线图:



2.2.2 光线采集电路设计

使用的探测元件是 LY-C1 光敏传感器。光敏电阻对周围环境光强最敏感，一般用来检测周围环境光强。输出状态灵敏度可调。光敏与 104 电位器串联分压，节点接到三极管基极，三极管将电阻的变化转换成电平的变化送给单片机，单片机读取电平，判断是白天还是夜晚。

光敏电阻又称光导管，常用的制作材料为硫化镉，另外还有硒、硫化铝、硫化铅和硫化铋等材料。这些制作材料具有在特定波长的光照射下，其阻值迅速减小的特性。这是由于光照产生的载流子都参与导电，在外加电场的作用下作漂移运动，电子奔向电源的正极，空穴奔向电源的负极，从而使光敏电阻器的阻值迅速下降。

光敏电阻器是利用半导体的光电导效应制成的一种电阻值随入射光的强弱而改变的电阻器，又称为光电导探测器；入射光强，电阻减小，入射光弱，电阻增大。

光敏电阻器一般用于光的测量、光的控制和光电转换（将光的变化转换为电的变化）。常用的光敏电阻器硫化镉光敏电阻器，它是由半导体材料制成的。光敏电阻器对光的敏感性（即光谱特性）与人眼对可见光（ $0.4 \sim 0.76$ ） μm 的响应很接近，只要人眼可感受的光，都会引起它的阻值变化。设计光控电路时，都用白炽灯泡（小电珠）光线或自然光线作控制光源，使设计大为简化。

通常，光敏电阻器都制成薄片结构，以便吸收更多的光能。当它受到光的照射时，半导体片（光敏层）内就激发出电子—空穴对，参与导电，使电路中电流增强。为了获得高的灵敏度，光敏电阻的电极常采用梳状图案，它是在一定的掩膜下向光电导薄膜上蒸镀金或钢等金属形成的。

以下图 4 和图 5 为光敏电阻资料图。



图 4 光敏电阻结构图

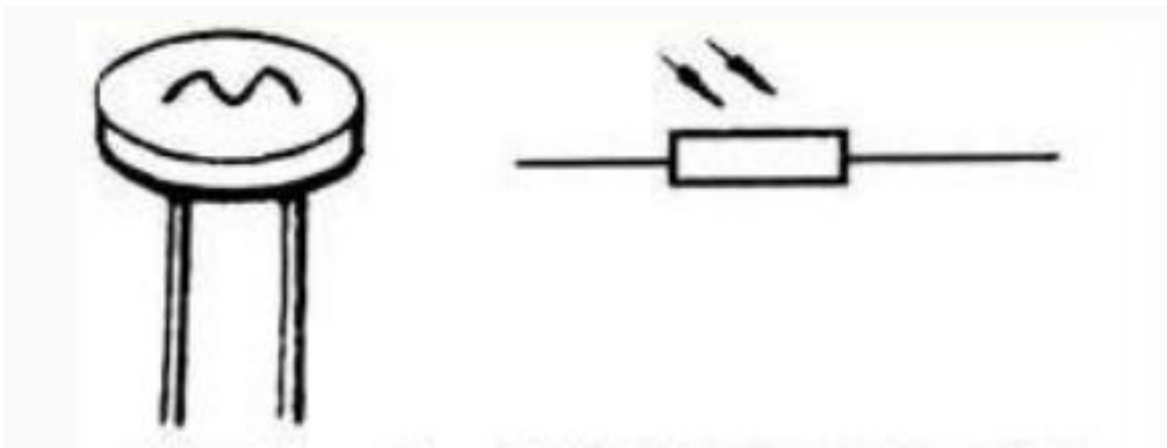


图 5 光敏电阻外形图和电路符号

根据光敏电阻的光谱特性，可分为三种光敏电阻器：紫外光敏电阻器、红外光敏电阻器、可见光光敏电阻器。本实验选用可见光光敏电阻器。如图 6 为光敏电阻的实验图。

光敏电阻的主要参数是：

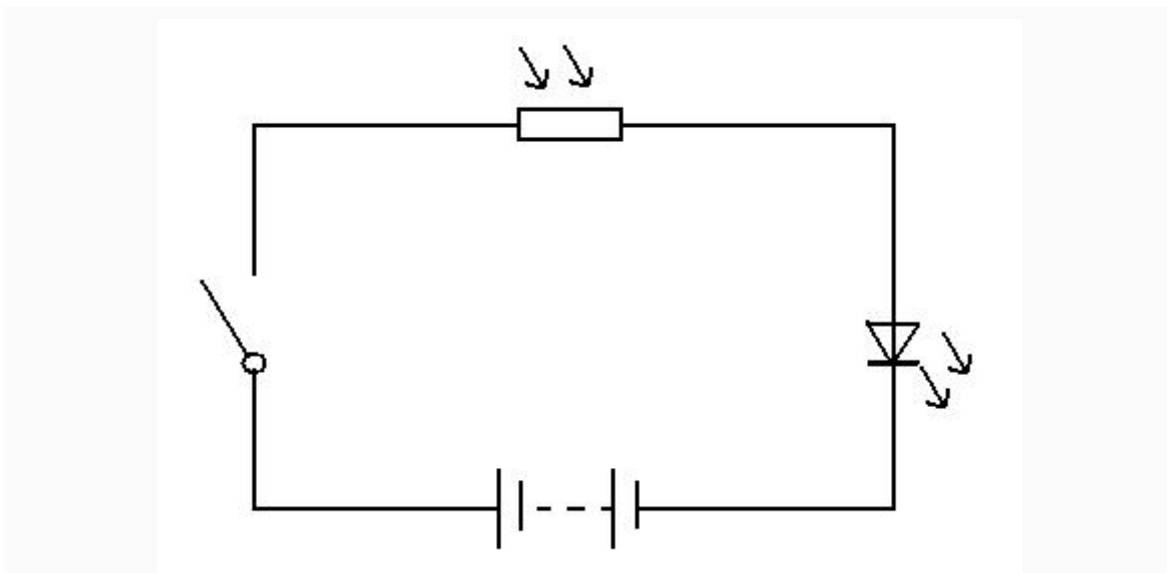


图 6 光敏电阻的实验图

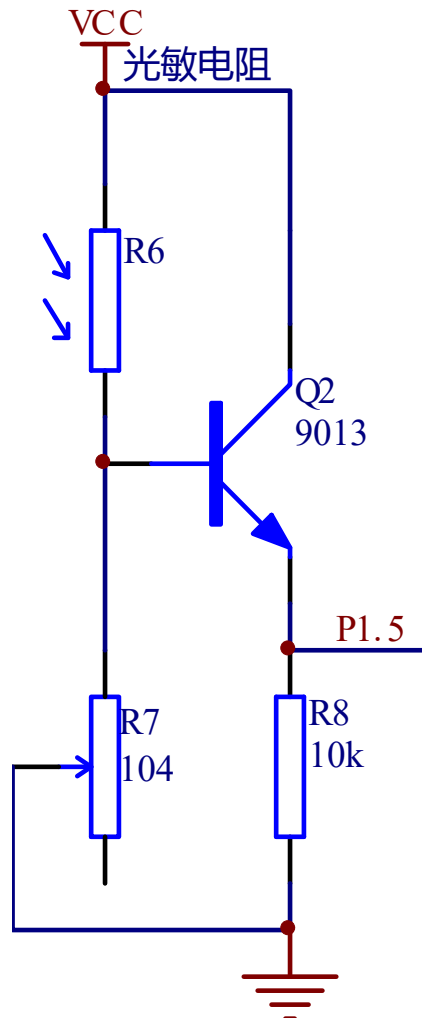
(1) 灵敏度。灵敏度是指光敏电阻不受光照射时的电阻值（暗电阻）与受光照射时的电阻值（亮电阻）的相对变化值。

(2) 光谱响应。光谱响应又称光谱灵敏度，是指光敏电阻在不同波长的单色光照射下的灵敏度。若将不同波长下的灵敏度画成曲线，就可以得到光谱响应的曲线。

(3) 光照特性。光照特性指光敏电阻输出的电信号随光照度而变化的特性。从光敏电阻的光照特性曲线可以看出，随着的光照强度的增加，光敏电阻的阻值开始迅速下降。若进一步增大光照强度，则电阻值变化减小，然后逐渐趋向平缓。在大多数情况下，该特性为非线性。

(4) 伏安特性曲线。伏安特性曲线用来描述光敏电阻的外加电压与光电流的关系，对于光敏器件来说，其光电流随外加电压的增大而增大。

光线传感器连线图：



2.2.3 温度传感器电路设计

在现代检测技术中，传感器占据着不可动摇的重要位置。主机对数据的处理能力已经相当的强，但是对现实世界中的模拟量却无能为力。如果没有各种精确可靠的传感器对非电量和模拟信号进行检测并提供可靠的数据，那计算机也无法发挥他应有的作用。传感器把非电量转换为电量，经过放大处理后，转换为数字量输入计算机，由计算机对信号进行分析处理。从而传感器技术与计算机技术结合起来，对自动化和信息化起重要作用。

采用各种传感器和微处理技术可以对各种工业参数及工业产品进行测控及检验，准确测量产品性能，及时发现隐患。为提高产品质量、改进产品性能，防止事故发生提供必要的信息和更可靠的数据。由于系统的工作环境比较恶劣，且对测量要求比较高，所以选择合适的传感器很重要。目前，国际上新型温度传感器正从模拟式向数字式、从集成化向智能化和网络化的方向飞速发展。智能温度传感器 DS18B20 正是朝着高精度、多功能、总线标准化、高可靠性及安全性、开发虚拟传感器和网络传感器、

研制单片测温系统等高科技的方向迅速发展。因此，智能温度传感器 DS18B20 作为温度测量装置已广泛应用于人民的日常生活和工农业生产中。

美国 DALLAS 公司生产的 DS18B20 可组网数字温度传感器芯片外加不锈钢保护管封装而成，具有耐磨耐碰，体积小，使用方便，封装形式多样，适用于各种狭小空间设备数字测温和控制领域。有独特的单线接口方式，DS1820 在与微处理器连接时仅需要一条口线即可实现微处理器与 DS1820 的双向通讯；其测温范围 $-55^{\circ}\text{C} \sim +125^{\circ}\text{C}$ ，固有测温分辨率 0.5°C ；支持多点组网功能；多个 DS1820 可以并联在唯一的三线上，实现多点测温；工作电源为 $3\sim 5\text{V}/\text{DC}$ ；在使用中不需要任何外围元件。

DS18B20 的性能特点如下：

- (1) 采用 DALLAS 公司独特的单线接口方式：DS18B20 与微处理器连接时仅需要一条口线即可实现微处理器与 DS18B20 的双向通讯；
- (2) 在使用中不需要任何外围元件；
- (3) 可用数据线供电，供电电压范围： $+3.0\text{V} \sim +5.5\text{V}$ ；
- (4) 测温范围： $-55 \sim +125^{\circ}\text{C}$ 。固有测温分辨率为 0.5°C 。当在 $-10^{\circ}\text{C} \sim +85^{\circ}\text{C}$ 范围内，可确保测量误差不超过 0.5°C ，在 $-55 \sim +125^{\circ}\text{C}$ 范围内，测量误差也不超过 2°C ；
- (5) 通过编程可实现 9~12 位的数字读数方式；
- (6) 用户可自设定非易失性的报警上下限值；
- (7) 支持多点的组网功能，多个 DS18B20 可以并联在唯一的三线上，实现多点测温
- (8) 负压特性，即具有电源反接保护电路。当电源电压的极性反接时，能保护 DS18B20 不会因发热而烧毁，但此时芯片无法正常工作；
- (9) DS18B20 的转换速率比较高，进行 9 位的温度值转换只需 93.75ms ；
- (10) 适配各种单片机或系统；
- (11) 内含 64 位激光修正的只读存储 ROM，扣除 8 位产品系列号和 8 位循环冗余校验码(CRC)之后，产品序号占 48 位。出厂前产品序号存入其 ROM 中。在构成大型温控系统时，允许在单线总线上挂接多片 DS18B20。

2.2.4 DS18B20 引脚介绍

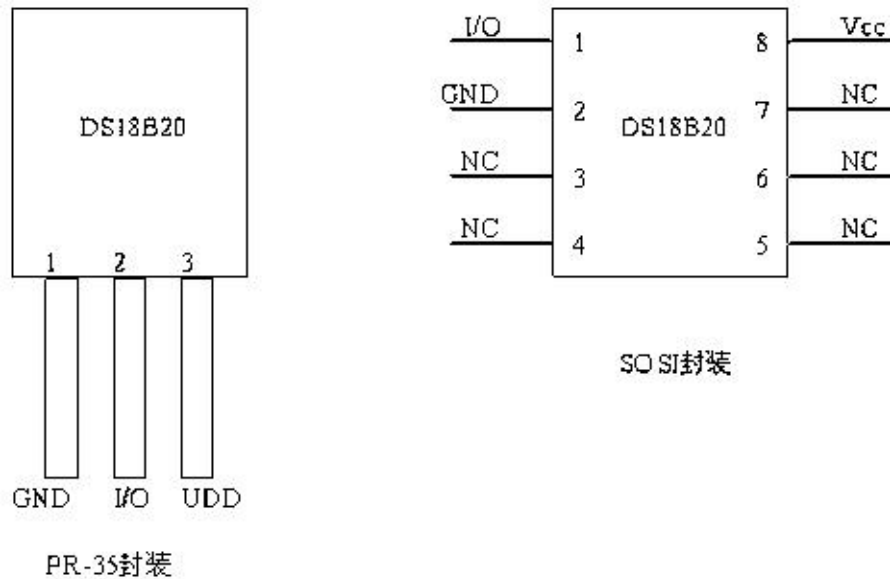


图 3: DS18B20 引脚

各引脚功能为：I/O 为数据输入/输出端（即单线总线），它属于漏极开路输出，外接上拉电阻后，常态下呈高电平。UDD 是可供选用的外部电源端，不用时接地，GND 为地，NC 空脚。

2.2.5 DS18B20 的内部结构

DS18B20 的内部结构主要包括 7 部分：寄生电源、温度传感器、64 位激光 (loser) ROM 与单线接口、高速暂存器（即便筏式 RAM，用于存放中间数据）、TH 触发寄存器和 TL 触发寄存器，分别用来存储用户设定的温度上下限值、存储和控制逻辑、位循环冗余校验码 (CRC) 发生器。

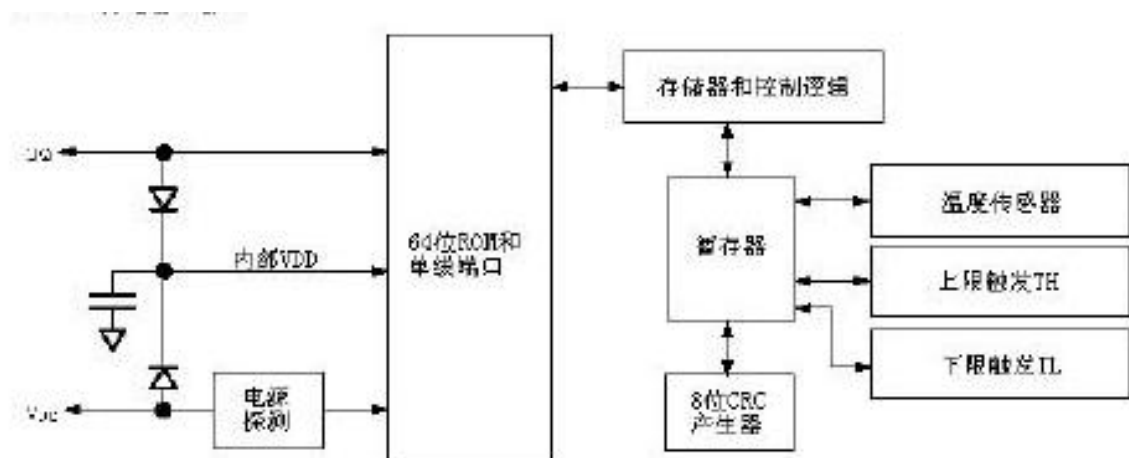
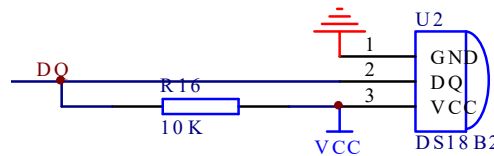


图 4: DS18B20 内部结构

温度传感器原理图:



2.2.6 烟雾传感器电路设计

此次设计是针对单片机原理及其应用展开的。其中包含了大学四年中所学到的相关知识，运用所学的传感器技术，单片机技术去设计基于单片机的烟雾报警系统。烟雾报警器的最基本组成部分包括：烟雾传感电路、模数转换电路、单片机控制电路。单片机紧紧联系着传感器和报警电路设备，给烟雾报警器设定一个值，当外界环境达到预先设定的数值时，烟雾传感器把被测的物理量作为输入参数，转换为电量输出。为了简化整个系统的设计在本设计中不采用前置放大器而是选择数值符合 A/D 转换器的输入等级。模数转换电路将从烟雾传感电路送出的模拟信号转换成单片机可识别的数字信号后送入单片机。这里选择单片机的 P1.0 为输入方式，接收到信号的单片机经过程序的设定会由 P2.0 作为单片机的输出电路，单片机对该数字信号进行滤波处理，并对处理后的数据进行分析，是否大于或等于某个预设值，如果大于则启动报警电路发出报警声音，反之则为正常状态。烟雾传感器及单片机是可燃烟雾检测报警器的两大核心，根据报警器功能的需要，选择合适、精确、经济的烟雾传感器及单片机芯片是至关重要的。

2.2.7 MQ-2 型烟雾传感器的工作原理

半导体烟雾传感器包括用氧化物半导体陶瓷材料作为敏感体制作的烟雾传感器以及用单晶半导体器件制作的烟雾传感器。按敏感机理分类，可分为电阻型和非电阻型。半导体气敏元件也有 N 型和 P 型之分。N 型在检测时阻值随烟雾浓度的增大而减小；P 型阻值随烟雾浓度的增大而增大。

本设计中采用的 MQ-2 型烟雾传感器属于二氧化锡半导体气敏材料，属于表面离子式 N 型半导体^[7]。当处于 200~300℃ 温度时，二氧化锡吸附空气中的氧，形成氧的负离子吸附，使半导体中的电子密度减少，从而使其电阻值增加。当与烟雾接触时，如果晶粒间界处的势垒受到该烟雾的调制而变化，就会引起表面电导率的变化。利用这一点就可以获得这种烟雾存在的信息。

遇到可燃烟雾（如 CH₄ 等）时，原来吸附的氧脱附，而由可燃烟雾以正离子状态吸附在二氧化锡半导体表面；氧脱附放出电子，烟雾以正离子状态吸附也要放出电子，从而使二氧化锡半导体导带电子密度增加，电阻值下降^[8]。而当空气中没有烟雾时，二氧化锡半导体又会自动恢复氧的负离子吸附，使电阻值升高到初始状态。这就是 MQ-2 型可燃性烟雾传感器检测可燃烟雾的基本原理。

2.2.8 MQ-2 型传感器的特性

a、MQ-2 型传感器的主要特点

MQ-2 型传感器主要有三个特点：（1）对天然气、液化石油气等烟雾有很高的灵敏度，尤其对烷类烟雾更为敏感。（2）具有良好的重复性和长期的稳定性。初始稳定，响应时间短，长时间工作性能好。（3）具有良好的抗干扰性，可准确排除有刺激性非可燃性烟雾的干扰信息，例如酒精和烟雾等。

b、MQ-2 型传感器的基本特性

MQ-2 型传感器的基本特性包括灵敏度特性、初期稳定特性、加热特性。

(1) 灵敏度特性：烟雾传感器在最佳工作条件下，接触同一种烟雾，其电阻值 R_S 随气体浓度变化的特性称之为灵敏度特性，用 K 表示。 $K=R_S / R_0$ (2-1) 式中， R_0 为烟雾传感器洁净空气条件下的电阻值^[9]， R_S 为烟雾传感器在一定浓度的检测烟雾中的电阻值。

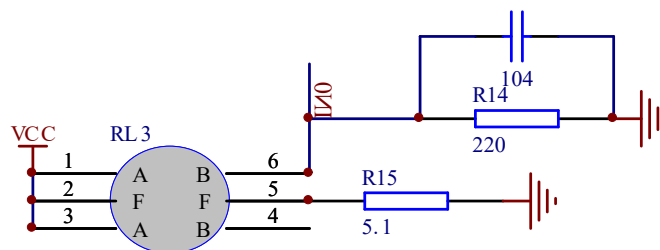
(2) 初期稳定特性：半导体烟雾传感器在不通电状态存放一段时间后，再通电时，器件并不能立即投入正常工作。这是因为烟雾传感器中的二氧化锡在不通电的状态下会吸附空气中的水蒸气，当再次通电时需要预热几分钟使水蒸气蒸发后，气敏电阻才能正常工作。再通电工作时气敏电阻值达到稳定时所需要的时间，定义为初期稳定时间。一般情况下，不通电时间越长，初期稳定时间也越长，当不通电存放时间达到 15 天左右时，初期稳定时间一般需要 5 分钟左右。

(3) 加热特性：半导体烟雾传感器一般要在较高的温度 ($200\sim 450^{\circ}\text{C}$) 下工作，所以需要对其加热。由于传感器一般工作在易燃易爆环境下，若加热丝直接与电源相接，当加热丝局部短路造成器件过热或放电时，可能引发事故。所以必须使用传感器生产厂家推荐的加热电压，使其工作在较安全的范围内。MQ-2 型烟雾传感器加热电压为 $5\pm 0.2\text{V}$ 。当加热丝断路时，由于热惰性缘故，烟雾传感器的气敏特性并不立即消失，此时检测必出现较大的误差。为避免出现这种情况，并及时发现气敏元件的故障，需要设计加热丝故障诊断报警电路。

c、MQ-2 型传感器的特性参数

- (1) 回路电压：(V_C) $5\sim 24\text{V}$
- (2) 取样电阻：(R_L) $0.1\sim 20\text{K}$
- (3) 加热电压：(V_H) $5\pm 0.2\text{V}$
- (4) 加热功率：(P) 约 750mW
- (5) 灵敏度：以甲烷为例 $R_0(\text{air})/R_S(0.1\%\text{CH}_4) > 5$
- (6) 响应时间： $T_{\text{res}} < 10$ 秒
- (7) 恢复时间： $T_{\text{rec}} < 30$ 秒

传感器电路：



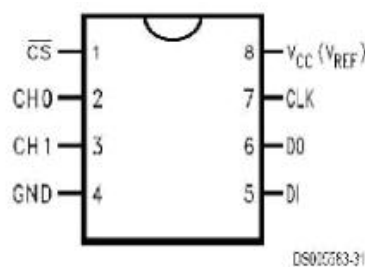
2.3 AD 电路设计

ADC0832 是美国国家半导体公司生产的一种 8 位分辨率、双通道 A/D 转换芯片。由于它体积小，兼容性强，性价比高而深受单片机爱好者及企业欢迎，其目前已经有非常高的普及率。学习并使用 ADC0832 可是使我们了解 A/D 转换器的原理，有助于我们单片机技术水平的提高。ADC0832 具有以下特点：

- 8 位分辨率；
- 双通道 A/D 转换

- 输入输出电平与 TTL/CMOS 相兼容
- 5V 电源供电时输入电压在 0-5V 之间
- 工作频率为 250KHZ，转换时间为 32 μ S
- 一般功耗仅为 15mW
- 8P、14P—DIP（双列直插）、PICC 多种封装
- 商用级芯片温宽为 0°C ~ +70°C，工业级芯片温宽为 -40°C ~ +85°C

芯片顶视图：



芯片接口说明：

- CS_ 片选使能，低电平芯片使能。
 - CH0 模拟输入通道 0，或作为 IN+/-使用。
 - CH1 模拟输入通道 1，或作为 IN+/-使用。
 - GND 芯片参考 0 电位（地）。
 - DI 数据信号输入，选择通道控制。
 - DO 数据信号输出，转换数据输出。
 - CLK 芯片时钟输入。
 - Vcc/REF 电源输入及参考电压输入（复用）。
- (5) $V_R(+)$ 和 $V_R(-)$ 为参考电压输入端。

2.3.1 ADC0832 的结构及转换原理

ADC0832 为 8 位分辨率 A/D 转换芯片，其最高分辨可达 256 级，可以适应一般的模拟量转换要求。其内部电源输入与参考电压的复用，使得芯片的模拟电压输入在 0~5V 之间。芯片转换时间仅为 32 μ S，据有双数据输出可作为数据校验，以减少数据误差，转换速度快且稳定性能强。独立的芯片使能输入，使多器件挂接和处理器控制变的更加方便。通过 DI 数据输入端，可以轻易的实现通道功能的选择。

正常情况下 ADC0832 与单片机的接口应为 4 条数据线，分别是 CS、CLK、DO、

DI。但由于 DO 端与 DI 端在通信时并未同时有效并与单片机的接口是双向的，所以电路设计时可以将 DO 和 DI 并联在一根数据线上使用。当 ADC0832 未工作时其 CS 输入端应为高电平，此时芯片禁用，CLK 和 DO/DI 的电平可任意。当要进行 A/D 转换时，须先将 CS 使能端置于低电平并且保持低电平直到转换完全结束。此时芯片开始转换工作，同时由处理器向芯片时钟输入端 CLK 输入时钟脉冲，DO/DI 端则使用 DI 端输入通道功能选择的数据信号。在第 1 个时钟脉冲的下沉之前 DI 端必须是高电平，表示启始信号。在第 2、3 个脉冲下沉之前 DI 端应输入 2 位数据用于选择通道功能。当此 2 位数据为“1”、“0”时，只对 CH0 进行单通道转换。当 2 位数据为“1”、“1”时，只对 CH1 进行单通道转换。当 2 位数据为“0”、“0”时，将 CH0 作为正输入端 IN+，CH1 作为负输入端 IN-进行输入。当 2 位数据为“0”、“1”时，将 CH0 作为负输入端 IN-，CH1 作为正输入端 IN+进行输入。到第 3 个脉冲的下沉之后 DI 端的输入电平就失去输入作用，此后 DO/DI 端则开始利用数据输出 DO 进行转换数据的读取。从第 4 个脉冲下沉开始由 DO 端输出转换数据最高位 DATA7，随后每一个脉冲下沉 DO 端输出下一位数据。直到第 11 个脉冲时发出最低位数据 DATA0，一个字节的的数据输出完成。也正是从此位开始输出下一个相反字节的数据，即从第 11 个字节的下沉输出 DATD0。随后输出 8 位数据，到第 19 个脉冲时数据输出完成，也标志着一次 A/D 转换的结束。最后将 CS 置高电平禁用芯片，直接将转换后的数据进行处理就可以了。

ADC0832 时序图：

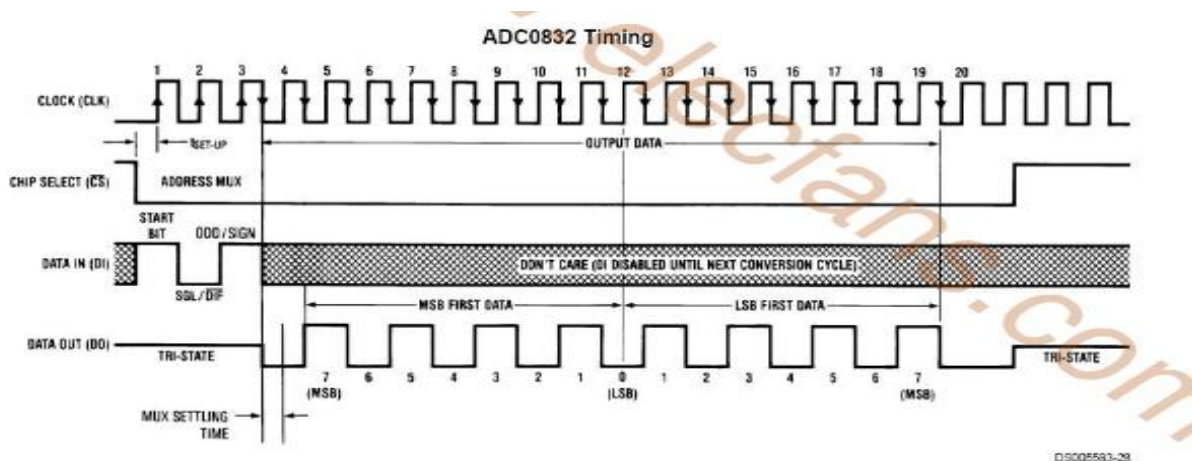


图 3-6

作为单通道模拟信号输入时 ADC0832 的输入电压是 0~5V 且 8 位分辨率时的电压精度为 19.53mV。如果作为由 IN+与 IN-输入的输入时，可是将电压值设定在某一

个较大范围之内，从而提高转换的宽度。但值得注意的是，在进行 IN+与 IN-的输入时，如果 IN-的电压大于 IN+的电压则转换后的数据结果始终为 00H。

2.3.2 ADC0832 连线图

ADC0832 与单片机的连线图如图 3-7:

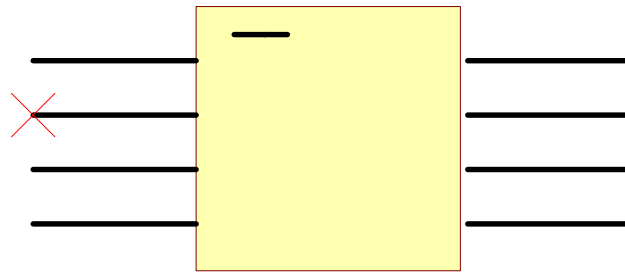


图3-7 ADC0809的连线图

2.4 单片机电路设计

STC89C51 为主要的中央处理系统，单片机是在集成电路芯片上集成了各种元件的微型计算机，这些元件包括中央处理器 CPU、数据存储器 RAM、程序存储器 ROM、定时/计数器、中断系统、时钟部件的集成和 I/O 接口电路。由于单片机具有体积小、价格低、可靠性高、开发应用方便等特点，因此在现代电子技术和工业领域应用较为广泛，在智能仪表中单片机是应用最多、最活跃的领域之一。在控制领域中，现如今人们更注意计算机的底成本、小体积、运行的可靠性和控制的灵活性。在各类仪器、仪表中引入单片机，使仪器仪表智能化，提高测试的自动化程度和精度，提高计算机的运算速度，简化仪器仪表的硬件结构，提高其性能价格比。

(一) STC89C51 主要功能、性能参数如下:

- (1) 内置标准 51 内核，机器周期：增强型为 6 时钟，普通型为 12 时钟；
- (2) 工作频率范围：0~40MHZ，相当于普通 8051 的 0~80MHZ；
- (3) STC89C51RC 对应 Flash 空间：4KB；
- (4) 内部存储器 (RAM)：256B；
- (5) 定时器\计数器：3 个 16 位；
- (6) 通用异步通信口 (UART) 1 个；
- (7) 中断源:8 个；
- (8) 有 ISP(在系统可编程) \IAP(在应用可编程),无需专用编程器\仿真器；
- (9) 通用 I/O 口：32\36 个；
- (10) 工作电压：3.8~5.5V；
- (11) 外形封装：40 脚 PDIP、44 脚 PLCC 和 PQFP 等。

(二) STC89C51 单片机的引脚说明:

VCC: 供电电压。

GND：接地。

P0 口：P0 口为一个 8 位漏级开路双向 I/O 口，每脚可吸收 8TTL 门电流。当 P1 口的管脚第一次写 1 时，被定义为高阻输入。P0 能够用于外部程序数据存储器，它可以被定义为数据/地址的第八位。在 FLASH 编程时，P0 口作为原码输入口，当 FLASH 进行校验时，P0 输出原码，此时 P0 外部必须被拉高。

P1 口：P1 口是一个内部提供上拉电阻的 8 位双向 I/O 口，P1 口缓冲器能接收输出 4TTL 门电流。P1 口管脚写入 1 后，被内部上拉为高，可用作输入，P1 口被外部下拉为低电平时，将输出电流，这是由于内部上拉的缘故。在 FLASH 编程和校验时，P1 口作为第八位地址接收。

P2 口：P2 口为一个内部上拉电阻的 8 位双向 I/O 口，P2 口缓冲器可接收，输出 4 个 TTL 门电流，当 P2 口被写“1”时，其管脚被内部上拉电阻拉高，且作为输入。并因此作为输入时，P2 口的管脚被外部拉低，将输出电流。这是由于内部上拉的缘故。P2 口当用于外部程序存储器或 16 位地址外部数据存储器进行存取时，P2 口输出地址的高八位。在给出地址“1”时，它利用内部上拉优势，当对外部八位地址数据存储器进行读写时，P2 口输出其特殊功能寄存器的内容。P2 口在 FLASH 编程和校验时接收高八位地址信号和控制信号。

P3 口：P3 口管脚是 8 个带内部上拉电阻的双向 I/O 口，可接收输出 4 个 TTL 门电流。当 P3 口写入“1”后，它们被内部上拉为高电平，并用作输入。作为输入，由于外部下拉为低电平，P3 口将输出电流（ILL）这是由于上拉的缘故。

P3.0 RXD（串行输入口）

P3.1 TXD（串行输出口）

P3.2 /INT0（外部中断 0）

P3.3 /INT1（外部中断 1）

P3.4 T0（记时器 0 外部输入）

P3.5 T1（记时器 1 外部输入）

P3.6 /WR（外部数据存储器写选通）

P3.7 /RD（外部数据存储器读选通）

P3 口同时为闪烁编程和编程校验接收一些控制信号。

I/O 口作为输入口时有两种工作方式，即所谓的读端口与读引脚。读端口时实际上并不从外部读入数据，而是把端口锁存器的内容读入到内部总线，经过某种运算或变换后再写回到端口锁存器。只有读端口时才真正地把外部的数据读入到内部总线。上面图中的两个三角形表示的就是输入缓冲器 CPU 将根据不同的指令分别发出读端口或读引脚信号以完成不同的操作。这是由硬件自动完成的，不需要我们操心，1 然后再实行读引脚操作，否则就可能读入出错，为什么看上面的图，如果不对端口置 1 端口锁存器原来的状态有可能为 0Q 端为 0Q[^]为 1 加到场效应管栅极的信号为 1，该场效应管就导通对地呈现低阻抗，此时即使引脚上输入的信号为 1，也会因端口的低阻抗而使信号变低使得外加的 1 信号读入后不一定是 1。若先执行置 1 操作，则可以使场效应管截止引脚信号直接加到三态缓冲器中实现正确的读入，由于在输入操作时还必须附加一个准备动作，所以这类 I/O 口被称为准双向口。STC89C51 的 P0/P1/P2/P3 口作为输入时都是准双向口。接下来让我们再看另一个问题，从图中可以看出这四个端口还有一个差别，除了 P1 口外 P0/P2/P3 口都还有其他的功能。

RST：复位输入。当振荡器复位器件时，要保持 RST 脚两个机器周期的高电平时间。

ALE/PROG：当访问外部存储器时，地址锁存允许的输出电平用于锁存地址的地位字节。在 FLASH 编程期间，此引脚用于输入编程脉冲。在平时，ALE 端以不变的频率周期输出正脉冲信号，此频率为振荡器频率的 1/6。因此它可用作对外部输出的脉冲或用于定时目的。然而要注意的是：每当用作外部数据存储器时，将跳过一个 ALE 脉冲。如想禁止 ALE 的输出可在 SFR8EH 地址上置 0。此时，ALE 只有在执行 MOVX，MOVC 指令是 ALE 才起作用。另外，该引脚被

略微拉高。如果微处理器在外部执行状态 ALE 禁止，置位无效。

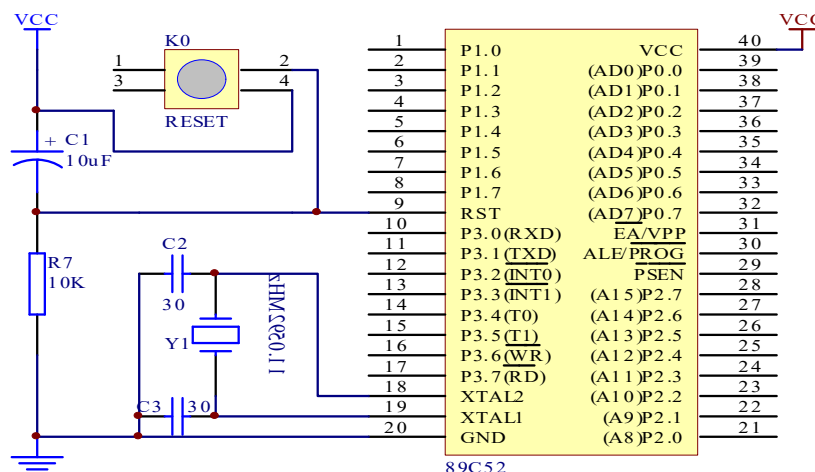
/PSEN：外部程序存储器的选通信号。在由外部程序存储器取指期间，每个机器周期两次/PSEN 有效。但在访问外部数据存储器时，这两次有效的/PSEN 信号将不出现。

/EA/VPP: 当/EA 保持低电平时, 则在此期间外部程序存储器 (0000H-FFFFH), 不管是否有内部程序存储器。注意加密方式 1 时, /EA 将内部锁定为 RESET; 当/EA 端保持高电平时, 此间内部程序存储器。在 FLASH 编程期间, 此引脚也用于施加 12V 编程电源 (VPP)。

XTAL1: 反向振荡放大器的输入及内部时钟工作电路的输入。

XTAL2: 来自反向振荡器的输出。

STC89C51 单片机的时钟信号通常有两种方式产生：一是内部时钟方式，二是外部时钟方式。在 STC89C51 单片机内部有一振荡电路，只要在单片机的 XTAL1 和 XTAL2 引脚外接石英晶体(简称晶振)，就构成了自激振荡器并在单片机内部产生时钟脉冲信号。电容的作用是稳定频率和快速起振，电容值在 5-30pF，典型值为 30pF。晶振 CYS 的振荡频率范围在 1.2-12MHz 间选择，典型值为 12MHz 和 11.0592MHz。



当在 STC89C51 单片机的 RST 引脚引入高电平并保持 2 个机器周期时,单片机内部就执行复位操作,按键手动复位有电平方式和脉冲方式两种。其中电平复位是通过 RST 端经过电阻与电源 VCC 接通而实现的。最小系统如图 10 所示。

图10 单片机最小系统电路

最小系统包括单片机及其所需的必要的电源、时钟、复位等部件，能使单片机始终处于正常的运行状态。电源、时钟等电路是使单片机能运行的必备条件，可以将最小系统作为应用系统的核心部分，通过对其进行存储器扩展、A/D 扩展等，使单片机完成较复杂的功能。

STC89C51 是片内有 ROM/EPROM 的单片机, 因此, 这种芯片构成的最小系统简单、可靠。用 STC89C52 单片机构成最小应用系统时, 只要将单片机接上时钟电路和复位电路即可, 结构如图 2-3 所示, 由于集成度的限制, 最小应用系统只能用作一些小型的控制单元。

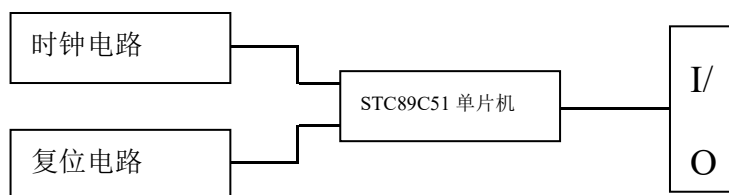


图 11 单片机最小系统原理框图

(1) 时钟电路

STC89C51 单片机的时钟信号通常有两种方式产生：一是内部时钟方式，二是外部时钟方式。内部时钟方式如图 2-4 所示。在 STC89C51 单片机内部有一振荡电路，只要在单片机的 XTAL1(18)和 XTAL2(19)引脚外接石英晶体(简称晶振)，就构成了自激振荡器并在单片机内部产生时钟脉冲信号。图中电容 C1 和 C2 的作用是稳定频率和快速起振，电容值在 5~30pF，典型值为 30pF。晶振 CYS 的振荡频率范围在 1.2~12MHz 间选择，典型值为 12MHz 和 6MHz。

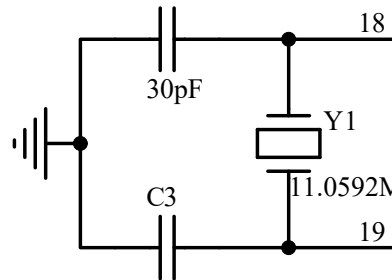


图 12 STC89C51 内部时钟电路

(2) 复位电路

当在 STC89C51 单片机的 RST 引脚引入高电平并保持 2 个机器周期时，单片机内部就执行复位操作(若该引脚持续保持高电平，单片机就处于循环复位状态)。

复位电路通常采用上电自动复位和按钮复位两种方式。

最简单的上电自动复位电路中上电自动复位是通过外部复位电路的电容充放电来实现的。只要 Vcc 的上升时间不超过 1ms,就可以实现自动上电复位。

除了上电复位外，有时还需要按键手动复位。本设计就是用的按键手动复位。按键手动复位有电平方式和脉冲方式两种。其中电平复位是通过 RST(9)端与电源 Vcc 接通而实现的。按键手动复位电路见图 2-5。时钟频率用 11.0592MHZ 时 C 取 10uF,R 取 10kΩ。

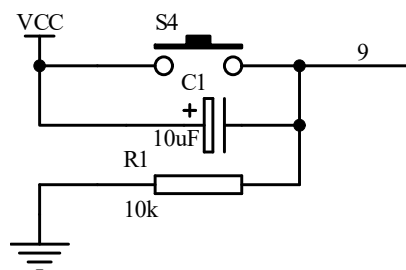
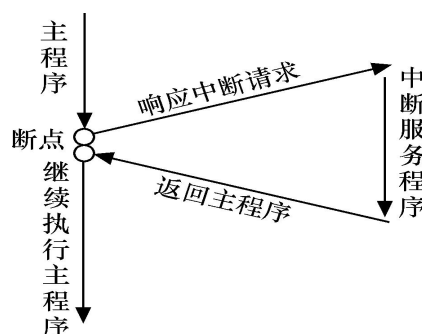


图 13 STC89C51 复位电路

(四) STC89C51 中断技术概述

中断技术主要用于实时监测与控制，要求单片机能及时地响应中断请求源提出的服务请求，并作出快速响应、及时处理。这是由片内的中断系统来实现的。当中断请求源发出中断请求时，如果中断请求被允许，单片机暂时中止当前正在执行的主程序，转到中断服务处理程序处理中断服务请求。中断服务处理程序处理完中断服务请求后，再回到原来被中止的程序之处(断点)，继续执行被中断的主程序。

如果单片机没有中断系统，单片机的大量时间可能会浪费在查询是否有服务请求发生的定时



查询操作上。采用中断技术完全消除了单片机在查询方式中的等待现象，大大地提高了单片机的工作效率和实时性。

2.5 报警电路设计

设计有报警电路，当有异常情况时，单片机的控制引脚就会输出低电平，利用三极管当做开关电路可以保护单片机，还可以起到放大电流的作用，当三极管基极为高电平时，发射极截止，为低电平时，发射极导通。报警模块如图 16 所示。

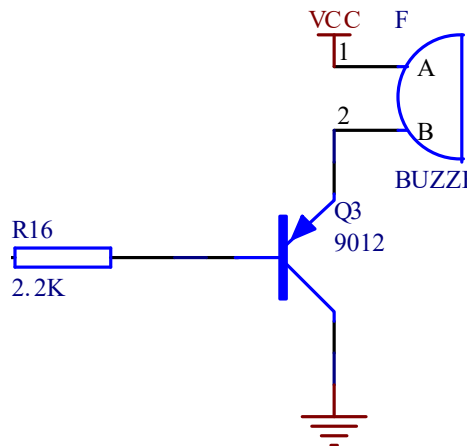


图 16 报警模块

2.6 液晶显示电路设计

液晶显示电路采用 LCD1602 液晶显示屏。其显示的内容为 16X2, 即可以显示两行，每行 16 个字符。

主要负责显示光度、烟雾浓度、窗户状态、温度和湿度。

LCD1602A 是一种工业字符型液晶，能够同时显示 16x02 即 32 个字符。（16 列 2 行）。在日常生活中，我们对液晶显示器并不陌生。液晶显示模块已作为很多电子产品的通过器件，如在计算器、万用表、电子表及很多家用电子产品中都可以看到，显示的主要是数字、专用符号和图形。在单片机的人机交流界面中，一般的输出方式有以下几种：发光管、LED 数码管、液晶显示器。发光管和 LED 数码管比较常用，软硬件都比较简单。

在单片机系统中应用晶液显示器作为输出器件有以下几个优点：

由于液晶显示器每一个点在收到信号后就一直保持那种色彩和亮度，恒定发光，而不像阴极射线管显示器（CRT）那样需要不断刷新新亮点。因此，液晶显示器画质高且不会闪烁。

液晶显示器都是数字式的，和单片机系统的接口更加简单可靠，操作更加方便。

液晶显示器通过显示屏上的电极控制液晶分子状态来达到显示的目的，在重量上比相同显示

面积的传统显示器要轻得多。

相对而言，液晶显示器的功耗主要消耗在其内部的电极和驱动 IC 上，因而耗电量比其它显示器要少得多。

(1) 引脚说明：

第 1 脚：VSS 为地电源。

第 2 脚：VDD 接 5V 正电源。

第 3 脚：VL 为液晶显示器对比度调整端，接正电源时对比度最弱，接地时对比度最高，对比度过高时会产生“鬼影”，使用时可以通过一个 10K 的电位器调整对比度。

第 4 脚：RS 为寄存器选择，高电平时选择数据寄存器、低电平时选择指令寄存器。

第 5 脚：R/W 为读写信号线，高电平时进行读操作，低电平时进行写操作。当 RS 和 R/W 共同为低电平时可以写入指令或者显示地址，当 RS 为低电平

R/W 为高电平时可以读忙信号，当 RS 为高电平 R/W 为低电平时可以写入数据。

第 6 脚：E 端为使能端，当 E 端由高电平跳变成低电平时，液晶模块执行命令。

第 7~14 脚：D0~D7 为 8 位双向数据线。

第 15 脚：背光源正极。

第 16 脚：背光源负极。

(2) 1602LCD 的 RAM 地址映射以及标准字库表

LCD1602 液晶模块内部的字符发生存储器已经存储了 160 个不同的点阵字符图形，这些字符图有：阿拉伯数字、英文字母的大小写、常用的符号、和日文假名等，每一个字符都有一个固定的代码，比如大写的英文字母“A”的代码是 01000001B (41H)，显示时模块把地址 41H 中的点阵字符图形显示出来，我们就能看到字母。

它的读写操作、屏幕和光标的操作都是通过指令编程来实现的（说明：1 为高电平，0 为低电平）。

指令 1：清显示，指令码 01H，光标复位到地址 00H 位置。

指令 2：光标复位，光标返回到地址 00H。

指令 3：光标和显示模式设置 I/D：光标移动方向，高电平右移，低电平左移。S：屏幕上所有文字是否左移或者右移。高电平表示有效，低电平则无效。

指令 4：显示开关控制。D：控制整体显示的开与关，高电平表示开显示，低电平表示关显示。C：控制光标的开与关，高电平表示有光标，低电平表示无光标。B：控制光标是否闪烁，高电平闪烁，低电平不闪烁。

指令 5：光标或显示移位 S/C：高电平时移动显示的文字，低电平时移动光标。

指令 6：功能设置命令 DL：高电平时为 4 位总线，低电平时为 8 位总线。N：低电平时为单行显示，高电平时为双行显示。F：低电平时显示 5X7 的点阵字符，高电平时显示 5x10 的点阵字符（有些模块是 DL：高电平时为 8 位总线，低电平时为 4 位总线）。

指令 7：字符发生器 RAM 地址设置。

指令 8：DDRAM 地址设置。

指令 9：读出忙信号和光标地址。BF 为忙标志位，高电平表示忙，此时模块不能接收命令或者数据，如果为低电平表示不忙，模块就能接收相应的命令或者数据。

指令 10：写数据。

指令 11：读数据。

液晶显示模块是一个慢显示器件，所以在执行每条指令之前一定要确认模块的忙标志为低电平，表示不忙，否则此指令失效。要显示字符时要先输入显示字符地址，也就是告诉模块在哪里

3.软件的设计

3.1 keil应用

Keil C51 是美国 Keil Software 公司出品的 51 系列兼容单片机 C 语言软件开发系统，与汇编相比，C 语言在功能上、结构性、可读性、可维护性上有明显的优势，因而易学易用。用过汇编语言后再使用 C 来开发，体会更加深刻。Keil C51 软件提供丰富的库函数和功能强大的集成开发调试工具，全 Windows 界面。另外重要的一点，只要看一下编译后生成的汇编代码，就能体会到 Keil C51 生成的目标代码效率非常之高，多数语句生成的汇编代码很紧凑，容易理解。在开发大型软件时更能体现高级语言的优势。下面详细介绍 Keil C51 开发系统各部分功能和使用。

Keil_c 软件界面如图 17:

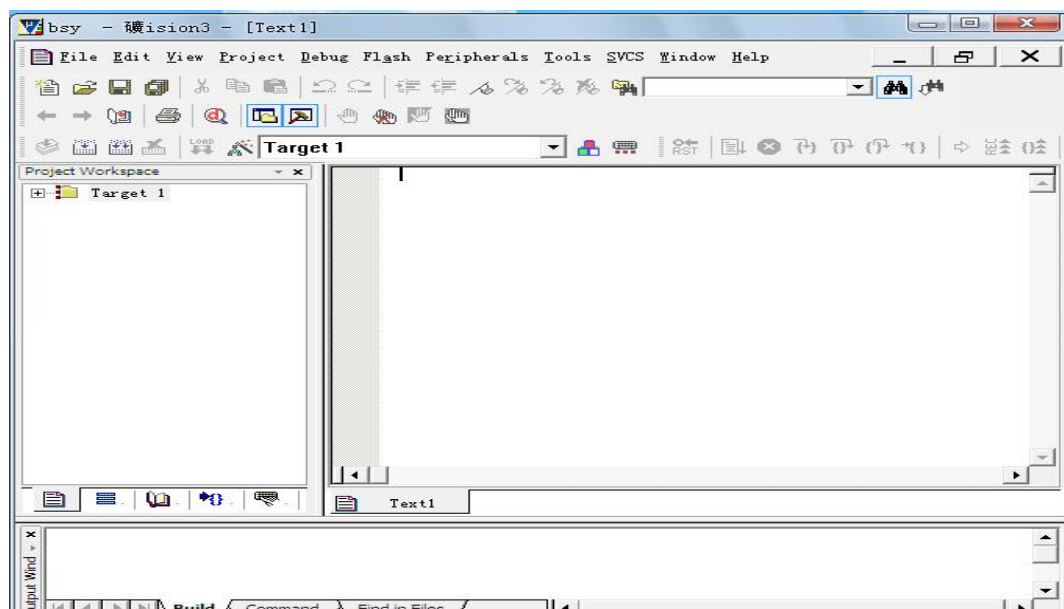


图 17 Keil_c 软件界面

该软件是一款集编程和仿真于一体的软件，它支持汇编、C 语言及二者的混合编程。

3.2 protel99se应用

Protel99SE 是 PORTEL 公司在 80 年代末推出的 EDA 软件。Protel99SE 是应用于 Windows9X/2000/NT 操作系统下的 EDA 设计软件，采用设计库管理模式，可以网设计，具有很强的数据交换能力和开放性及 3D 模拟功能，是一个 32 位的设计软件，可以完成电路原理图设计，印制电路板设计和可编程逻辑器件设计等工作，可以设计 32 个信号层，16 个电源—地层和 16 个机加工层。

Protel99SE 软件的特点:

- (1) 可生成 30 多种格式的电气连接网络表；
- (2) 强大的全局编辑功能；
- (3) 在原理图中选择一级器件，PCB 中同样的器件也将被选中；
- (4) 同时运行原理图和 PCB，在打开的原理图和 PCB 图间允许双向交叉查找元器件、引脚、网络
- (5) 既可以进行正向注释元器件标号（由原理图到 PCB），也可以进行反向注释（由 PCB 到原理图），以保持电气原理图和 PCB 在设计上的一致性；
- (6) 满足国际化设计要求（包括国标标题栏输出，GB4728 国标库）； * 方便易用的数模混合仿真（兼容 SPICE 3f5）；
- (7) 支持用 CUPL 语言和原理图设计 PLD，生成标准的 JED 下载文件； * PCB 可设计 32 个信号层，16 个电源-地层和 16 个机加工层；
- (8) 强大的“规则驱动”设计环境，符合在线的和批处理的设计规则检查；
- (9) 智能覆铜功能，覆铜可以自动重铺；
- (10) 提供大量的工业化标准电路板做为设计模版；
- (11) 放置汉字功能；
- (12) 可以输入和输出 DXF、DWG 格式文件，实现和 AutoCAD 等软件的数据交换；
- (13) 智能封装导航（对于建立复杂的 PGA、BGA 封装很有用）；
- (14) 方便的打印预览功能，不用修改 PCB 文件就可以直接控制打印结果；

3.3 DS18B20 的程序流程图

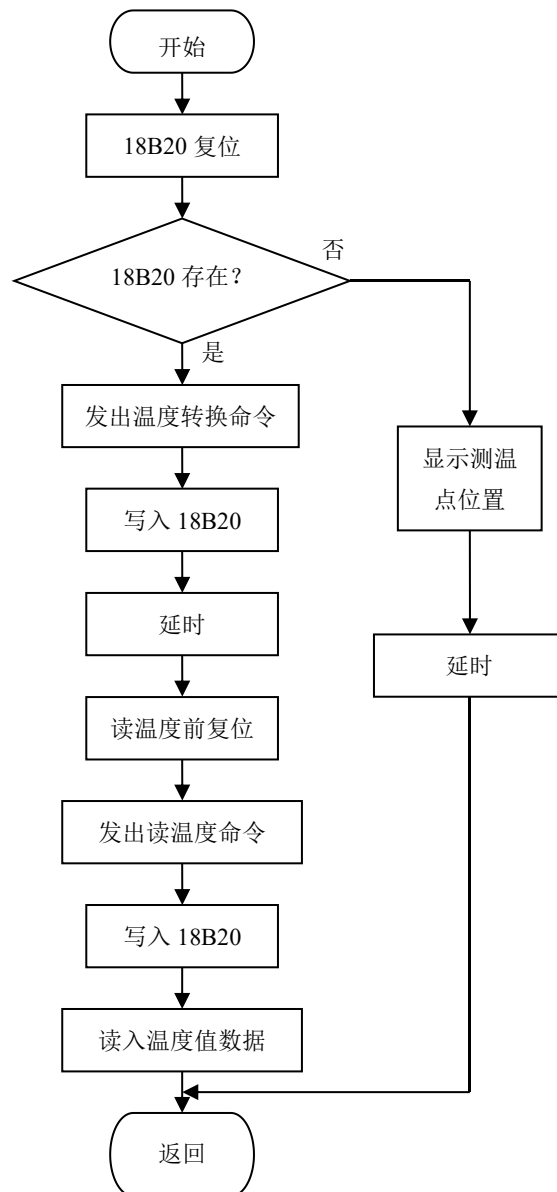


图 5 程序流程图

4.系统分析与调试

本设计是在 Keil C 环境下开发的，Keil C 软件支持 C 语言的编程及调试，运用方便，是做 C 语言毕业设计者的首选。设计的首要任务是安装和学习使用这个软件，在简单的学习和了解 Keil C 后，在编译完 Keil C 后，再运用 STC_ISP_V480 软件烧录到开发板上，实现实物与程序的连接。在烧录前要对 STC_ISP_V480 进行一些必要的设置。第一步：设置 MCU Type 为 STC89C51RC；

第二步：打开编写好并编译的程序文件，它是以.hex 为后缀的文件；第三步：选择对应的 COM 端口，（可在我的电脑的设备管理处查看 COM 选项）；第四步：点击 Download/下载，等提示请给 MCU 上电时，打开开发板上的开关，它就自行烧录了。

Keil C 程序运行如图 4-1 所示，下载图如图 4-2 所示：

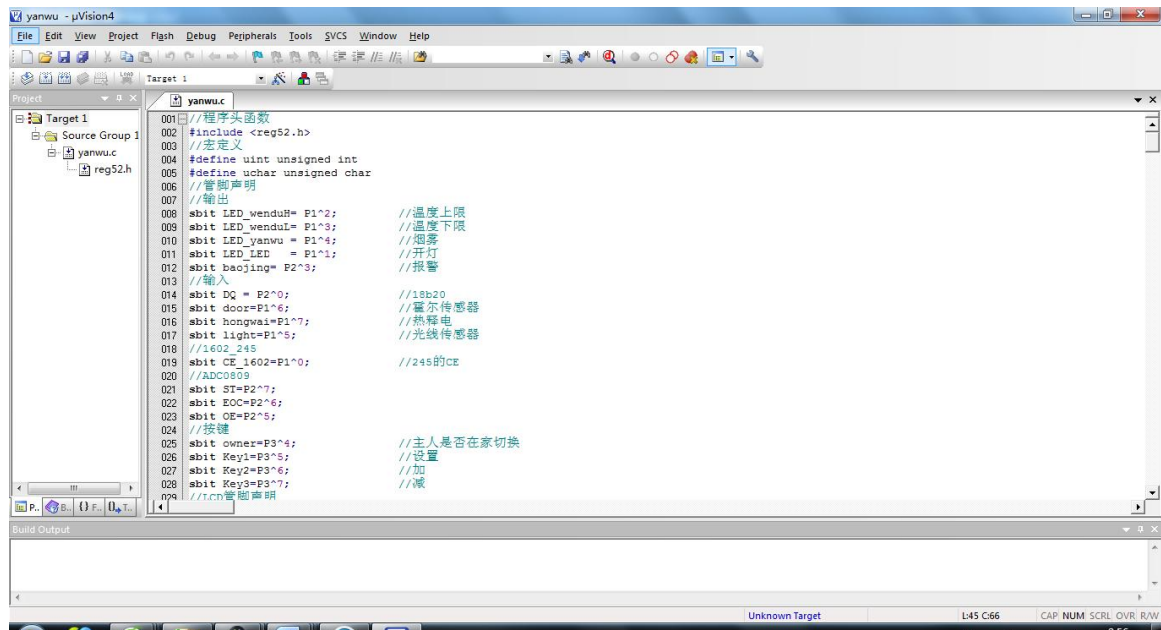


图 19 keil C 运行图

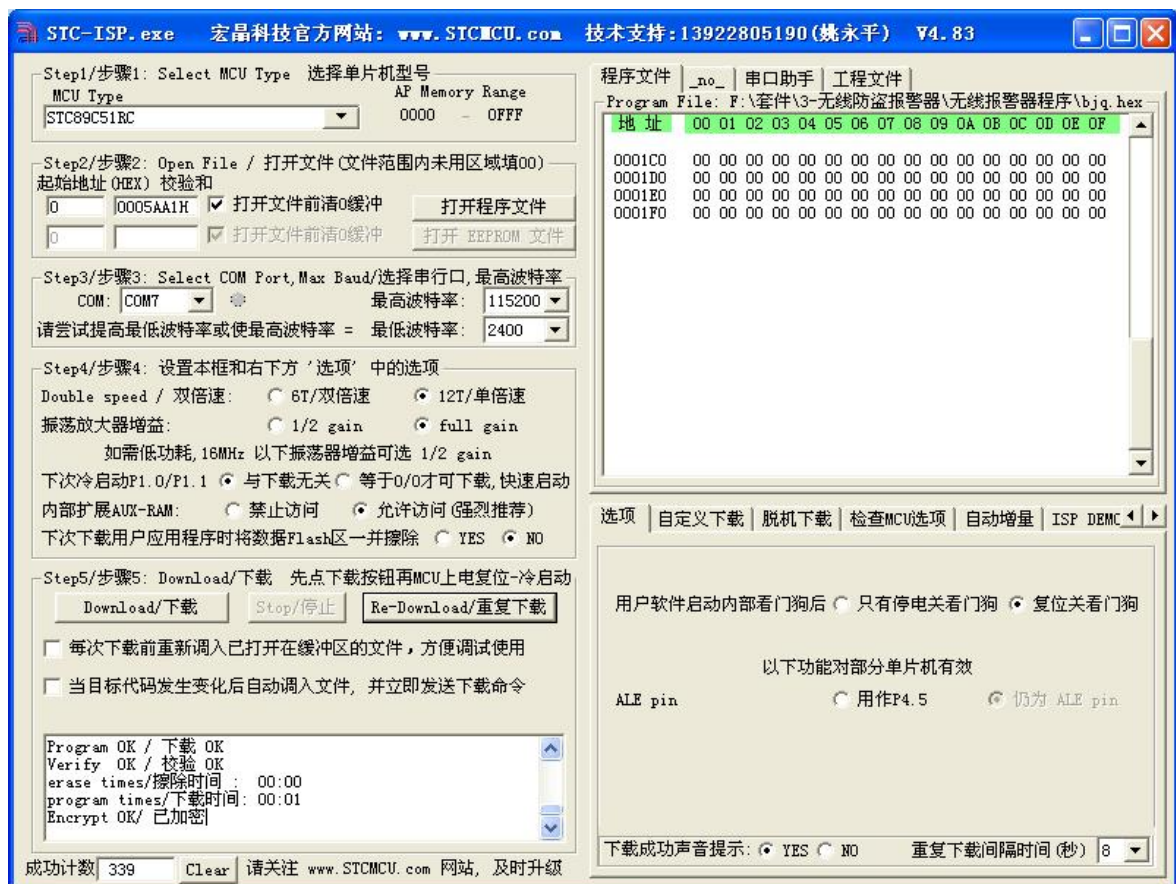


图 20 程序烧录运行图

结 论

通过本次毕业设计，使我对所学的单片机以及电路制图等方面的知识得到了巩固，并且有了进一步的深入了解。通过查阅和收集了大量的相关资料，通过计算和分析，终于将设计圆满的完成。

在设计过程中，让我发现仅仅有理论知识是完全不够的，实践的东西也是相当重要的。只有将理论和实践很好的相结合起来才能更好的完成自己的工作。这对我以后的学习和工作有了很大的帮助。

还有就是让我对报警器的制作流程也有了一定的了解。从得到指标开始，设计原理图，PCB图，焊板，调试。这里让我知道。对待要调试验证的掉路，一定要优先保证连接上的畅通，仔细检查是不是有虚焊，漏焊的存在。还有就是要注意我们采用的器件是不是符合设计的要求。最后还要通过通电之前的数据测量检查一下电路的安全情况。

通过这次设计让我发现了自己的很多不足之处，比如实际动手能力差等等。让我明白了自己需要提高的地方还很多，需要学习的也很多。

致谢

本文得以完成，首先要感谢我的指导老师，本论文的全部研究工作都是老师的精心指导和严格要求下完成的。老师的治学态度和渊博的学识对我的一生都会有重要的影响。在论文工作期间，无论是学习上，还是生活中，老师都给我以耐心的教诲和极大的帮助。老师踏实严谨的工作作风，渊博的学识，高效的办事效率，忘我的工作精神，让我获益匪浅。尤其是他积极开拓不断进取的精神，更是我终生学习的楷模。老师的平易近人也使我能随时与他进行沟通，提出开发过程中遇到的困难并得到解决。本课题能够顺利完成与老师的帮助和指导是分不开的，在此向他表示最诚挚的谢意。

感谢电子系老师们在学习上的帮助。感谢学习期间各位师兄、师姐、师弟、师妹们以及学友营造的良好的学习环境。感谢我的家人，正是她们的大力支持才使我能潜心于本课题的研究，对我完成学业起了重大的推动作用。对在百忙之中抽出时间评阅本论文的专家学者表示真诚的感谢！向所有关心，支持和帮助过我的老师、同学、亲人和朋友表示谢意！在即将完成学业之际，我还要衷心地感谢我的母校——XXXX 大学对我的培养！

学生:XXX

时间:2014 年 12 月 30 日

参考文献

- [1]赵负图等. 新型传感器集成电路应用手册[M]. 北京:人民邮电出版 2009. 9-1
- [2]彭同明, 徐学勤, 杨少华, 等. 单片机原理及应用[M]. 北京:中国电力出版社, 2007. 12-13.
- [3]张军, 宋涛. AVR 单片机 C 语言程序设计实例精粹[M]. 北京:电子工业出版社, 2009. 4-7.
- [4]刘坤, 宋戈, 赵红波, 等. 51 单片机 C 语言应用开发技术大全[M]. 北京:人民邮电出版社, 2008. 352-360.
- [5]张萌, 和湘, 姜斌, 等. 单片机应用系统开发综合实例[M]. 北京:清华大学出版
- [6]王守忠. 51 单片机典型模块设计实例导航[M]. 北京:人民邮电出版 2008. 4-7
- [7]求是科技等. 单片机典型模块设计实例导航[M]. 北京:人民邮电出版 2008. 7-1
- [8] 杜树春, 张体才. 单片机与外围器件借口实例详解[M]. 北京:中国电力出版社 2009. 1-1
- [9] 向忠宏等. 智能家居[M]. 北京:人民邮电出版社 2002. 6-1
- [10]韩江洪等. 智能家居系统与技术[M]. 安徽: 合肥工业大学出版社 2009. 1-1
- [11]孙育才. AT89S52 系列单片机及其应用[M]. 北京:清华大学出版社, 2005.
- [12]马淑华, 王凤文, 张美金. 单片机原理与接口技术[M]. 北京:北京邮电大学出版社, 2005.
- [13]万光毅, 严义. 单片机实验与实践教程[M]. 北京:北京航空航天大学出版社, 2004.
- [14]张洪润, 易涛. 单片机应用技术教程[M]. 北京:清华大学出版社, 2003.
- [15]吴选忠, 倪子伟. 基于 PC 机的新型智能火灾报警控制系统研究[J]. 低压电器现代建筑电气篇, 2008(4):46-50.
- [16]CELIK T, DEMIREL H, OZKARAMANLIH, et al. Fire detection using statistical color model in video sequences[J]. Journal of Visual Communication and Image Representation, 2007, 18(2):176-185.
- [17]KANG Ka. i A smoke model and its application for smoke management in an underground mass transit station[J]. Fire Safety Journal, 2007, 42(3):218-231.
- [18]MARBACHG, LOEPFEM, BRUPBACHER T. An image processing technique for fire detection in video images[J]. Fire Safety Journal, 2006, 41(2):285-289.

附录 3 源程序

```

#include <reg52.h>
#include <intrins.h>
#include <ADC0832.H>

#define uchar unsigned char    // 以后 unsigned char 就可以用 uchar 代替
#define uint unsigned int     // 以后 unsigned int 就可以用 uint 代替

sfr ISP_DATA = 0xe2;          // 数据寄存器
sfr ISP_ADDRH = 0xe3;         // 地址寄存器高八位
sfr ISP_ADDRL = 0xe4;         // 地址寄存器低八位
sfr ISP_CMD = 0xe5;           // 命令寄存器
sfr ISP_TRIG = 0xe6;           // 命令触发寄存器
sfr ISP_CONTR = 0xe7;         // 命令寄存器

sbit LcdRs_P = P2^5;          // 1602 液晶的 RS 管脚
sbit LcdRw_P = P2^6;          // 1602 液晶的 RW 管脚
sbit LcdEn_P = P2^7;          // 1602 液晶的 EN 管脚

sbit KeySet_P = P2^0;          // “设置” 按键的管脚
sbit KeyDown_P = P2^1;        // “减” 按键的管脚
sbit KeyUp_P = P2^2;          // “加” 按键的管脚
sbit Buzzer_P = P2^3;         // 蜂鸣器的管脚
sbit Led_P = P2^3;            // LED 报警灯的管脚
sbit DQ = P3^7;               // 温度传感器的引脚定义

int Temp1;
int Temp_L=200;
int Temp_H=500;

uchar PH_H=90;
uchar TD_H=40;

uchar menu_1;                //菜单设计的变量

int temp;                    // 保存温度传感器测量到的结果

/*****
// 单片机内部 EEPROM 不使能
*****/
void ISP_Disable()

```

```

{
    ISP_CONTR = 0;
    ISP_ADDRH = 0;
    ISP_ADDRL = 0;
}

/*****/
// 从单片机内部 EEPROM 读一个字节，从 0x2000 地址开始
/*****/
unsigned char EEPROM_Read(unsigned int add)
{
    ISP_DATA = 0x00;
    ISP_CONTR = 0x83;
    ISP_CMD = 0x01;
    ISP_ADDRH = (unsigned char)(add>>8);
    ISP_ADDRL = (unsigned char)(add&0xff);
    // 对 STC89C51 系列来说，每次要写入 0x46，再写入 0xB9,ISP/IAP 才会生效
    ISP_TRIG = 0x46;
    ISP_TRIG = 0xB9;
    _nop_();
    ISP_Disable();
    return (ISP_DATA);
}

/*****/
// 往单片机内部 EEPROM 写一个字节，从 0x2000 地址开始
/*****/
void EEPROM_Write(unsigned int add,unsigned char ch)
{
    ISP_CONTR = 0x83;
    ISP_CMD = 0x02;
    ISP_ADDRH = (unsigned char)(add>>8);
    ISP_ADDRL = (unsigned char)(add&0xff);
    ISP_DATA = ch;
    ISP_TRIG = 0x46;
    ISP_TRIG = 0xB9;
    _nop_();
    ISP_Disable();
}

/*****/

```

```
// 擦除单片机内部 EEPROM 的一个扇区
// 写 8 个扇区中随便一个的地址，便擦除该扇区，写入前要先擦除
/*****/
```

```
void Sector_Erase(unsigned int add)
{
    ISP_CONTR = 0x83;
    ISP_CMD   = 0x03;
    ISP_ADDRH = (unsigned char)(add>>8);
    ISP_ADDRL = (unsigned char)(add&0xff);
    ISP_TRIG  = 0x46;
    ISP_TRIG  = 0xB9;
    _nop_();
    ISP_Disable();
}
```

```
/*****/
// 毫秒级的延时函数，time 是要延时的毫秒数
/*****/
```

```
void DelayMs(uint time)
{
    uint i,j;
    for(i=0;i<time;i++)
        for(j=0;j<112;j++);
}
```

```
/*****/
// 延时 15 微秒
/*****/
```

```
void Delay15us(void)
{
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    _nop_();
}
```

```

    _nop_();
    _nop_();
    _nop_();
    _nop_();
}

/*****/
// 1602 液晶写命令函数，cmd 就是要写入的命令
/*****/
void LcdWriteCmd(uchar cmd)
{
    LcdRs_P = 0;
    LcdRw_P = 0;
    DelayMs(2);
    LcdEn_P = 1;
    DelayMs(2);
    LcdEn_P = 0;
}

/*****/
// 1602 液晶写数据函数，dat 就是要写入的数据
/*****/
void LcdWriteData(uchar dat)
{
    LcdRs_P = 1;
    LcdRw_P = 0;
    LcdEn_P = 0;
    P0=dat;
    DelayMs(2);
    LcdEn_P = 1;
    DelayMs(2);
    LcdEn_P = 0;
}

/*****/
// 1602 液晶初始化函数
/*****/
void LcdInit()
{
    LcdWriteCmd(0x38);          // 16*2 显示，5*7 点阵，8 位数据口
    LcdWriteCmd(0x0C);          // 开显示，不显示光标

```

```

        LcdWriteCmd(0x06);           // 地址加 1，当写入数据后光标右移
        LcdWriteCmd(0x01);           // 清屏
    }

/*****
// 液晶光标定位函数
*****/
void LcdGotoXY(uchar line,uchar column)
{
    // 第一行
    if(line==0)
        LcdWriteCmd(0x80+column);
    // 第二行
    if(line==1)
        LcdWriteCmd(0x80+0x40+column);
}

/*****
// 液晶输出字符串函数
*****/
void LcdPrintStr(uchar *str)
{
    while(*str!='\0')
        LcdWriteData(*str++);
}

/*****
// 液晶输出数字
*****/
void LcdPrintNum(uint num)
{
    LcdWriteData(num/100+0x30);           // 百位
    LcdWriteData(num%100/10+0x30);       // 十位
    LcdWriteData(num%10+0x30);           // 个位
}
void LcdPrintNum_Ponit(uint num)
{
    LcdWriteData(num/100+0x30);           // 百位
    LcdWriteData(num%100/10+0x30);       // 十位
    LcdWriteData('.');
}

```

```

        LcdWriteData(num%10+0x30);                                // 个位
    }

    /**
     * 在液晶上显示温度
     */
    void LcdPrintTemp(int temp)
    {
        if(temp<0)
        {
            LcdWriteData('-');
            // 负号
            temp=0-temp;
            // 负数转为正数
        }
        if(temp>999)
        {
            // LcdWriteData(temp/1000+0x30);                // 百位
        }
        else
        {
            LcdWriteData(temp%1000/100+0x30);                // 十位
            LcdWriteData(temp%100/10+0x30);                  // 个位
            LcdWriteData('.');
            // 小数点
            LcdWriteData(temp%10+0x30);                        // 小数后一位
            LcdWriteData('C');
            LcdWriteData(' ');
        }
        Temp1=(temp%1000/100)*100+(temp%100/10)*10+(temp%10);
    }

    /**
     * 复位 DS18B20（初始化）
     */
    void DS18B20_ReSet(void)
    {
        uchar i;
        DQ=0;
    }

```

```

        i=240;
        while(--i);
        i=30;
        while(--i);
        while(~DQ);
        i=4;
        while(--i);
    }

    /**
     * 向 DS18B20 写入一个字节
     */
    void DS18B20_WriteByte(uchar dat)
    {
        uchar j;
        uchar btmp;

        for(j=0;j<8;j++)
        {
            btmp=0x01;
            btmp=btmp<<j;
            btmp=btmp&dat;

            if(btmp>0)                // 写 1
            {
                DQ=0;
                Delay15us();
                DQ=1;
                Delay15us();
                Delay15us();
                Delay15us();
                Delay15us();
            }
            else                        // 写 0
            {
                DQ=0;
                Delay15us();
                Delay15us();
                Delay15us();
                Delay15us();
                DQ=1;
                Delay15us();
            }
        }
    }

```

```

    }
}

/*****
// 读取温度值
*****/
int DS18B20_ReadTemp(void)
{
    uchar j;
    int b,temp=0;

    DS18B20_ReSet();
    // 产生复位脉
    DS18B20_WriteByte(0xcc);                // 忽略 ROM 指令
    DS18B20_WriteByte(0x44);                // 启动温度转换指令

    DS18B20_ReSet();
    // 产生复位脉
    DS18B20_WriteByte(0xcc);                // 忽略 ROM 指令
    DS18B20_WriteByte(0xbe);                // 读取温度指令

    for(j=0;j<16;j++)
    // 读取温度数量
    {
        DQ=0;
        _nop_();
        _nop_();
        DQ=1;
        Delay15us();
        b=DQ;
        Delay15us();
        Delay15us();
        Delay15us();
        b=b<<j;
        temp=temp|b;
    }

    temp=temp*0.0625*10;                    // 合成温度值并
放大 10 倍
    return (temp);
    // 返回检测到的温度值
}

void LcdPrint_PH_H(uchar hang,uchar add,uchar date)

```

```

{
    if(hang==0)
        LcdWriteCmd(0x80+add);
    else
        LcdWriteCmd(0x80+0x40+add);
    LcdWriteData(date/100+0x30); // 百位
    LcdWriteData(date%100/10+0x30); // 十位
    LcdWriteData('.');
    LcdWriteData(date%10+0x30); // 个位
}
void LcdPrint_PH(uchar hang,uchar add,uchar date)
{
    if(hang==0)
        LcdWriteCmd(0x80+add);
    else
        LcdWriteCmd(0x80+0x40+add);
    LcdWriteData('G');
    LcdWriteData('X');
    LcdWriteData(':');
    LcdWriteData(date/100+0x30); // 百位
    LcdWriteData(date%100/10+0x30); // 十位
    LcdWriteData(date%10+0x30); // 个位
    LcdWriteData(' ');
}
void LcdPrint_TD(uchar hang,uchar add,uchar date)
{
    if(hang==0)
        LcdWriteCmd(0x80+add);
    else
        LcdWriteCmd(0x80+0x40+add);
    LcdWriteData('Y');
    LcdWriteData('W');
    LcdWriteData(':');
    LcdWriteData(date/100+0x30); // 百位
    LcdWriteData(date%100/10+0x30); // 十位
    LcdWriteData(date%10+0x30); // 个位
    LcdWriteData('%');
    LcdWriteData(' ');
}
/*****
// 按键扫描
*****/
void KeyScanf()
{

```

```

if(KeySet_P==0)
    // 判断是否有按键按下
{
    DelayMs(10);
    // 消除按键按下的抖动
    if(KeySet_P==0)
    {
        while(!KeySet_P);
    // 等待按键释放
        menu_1++;
        if(menu_1==4)
            menu_1=0;
    }
}

while(menu_1!=0)
{
    if(KeySet_P==0)
        // 判断是否有按键按下
        {
            DelayMs(10);
            // 消除按键按下的抖动
            if(KeySet_P==0)
            {
                while(!KeySet_P);
            // 等待按键释放
                menu_1++;
                if(menu_1==4)
                    menu_1=0;
            }
        }
    if(menu_1==1)
    {
        if(KeyDown_P==0)
            // 报警值减的处理
        {
            DelayMs(10);
            // 消除按键按下的抖动
            if(KeyDown_P==0)
            {
                while(!KeyDown_P); // 等待按键释放
                Temp_H--;
            }
        }
        if(KeyUp_P==0)
            // 报警值加处理
        {
            DelayMs(10);

```

```

        if(KeyUp_P==0)
        {
            while(!KeyUp_P);
            Temp_H++;
        }
    }
    if(menu_1==2)
    {
        if(KeyDown_P==0)                // 报警值减的处理
    {
        DelayMs(10);                    // 消除按键按下的抖动
        if(KeyDown_P==0)
        {
            while(!KeyDown_P); // 等待按键释放
            TD_H--;
        }
    }
        if(KeyUp_P==0)                // 报警值加处理
    {
        DelayMs(10);
        if(KeyUp_P==0)
        {
            while(!KeyUp_P);
            TD_H++;
        }
    }
    if(menu_1==3)
    {
        if(KeyDown_P==0)                // 报警值减的处理
    {
        DelayMs(10);                    // 消除按键按下的抖动
        if(KeyDown_P==0)
        {
            while(!KeyDown_P); // 等待按键释放
            PH_H--;
        }
    }
        if(KeyUp_P==0)                // 报警值加处理
    {
        DelayMs(10);
        if(KeyUp_P==0)
        {

```

```

                                while(!KeyUp_P);
                                PH_H++;
                                }
                                }
                                }

if(menu_1==1)
{
    LcdGotoXY(0,0);
晶恢复测量时的内容显示
                                LcdPrintStr("    Temp Max set    ");
                                LcdGotoXY(1,0);
                                LcdPrintStr("    ");
                                LcdGotoXY(1,2);
                                LcdPrintStr("T:");
                                LcdGotoXY(1,4);
                                // 定位到第 0 行第 7 列
                                LcdPrintTemp(Temp_H);
                                // 显示当前的温度值
                                LcdGotoXY(1,9);
                                LcdPrintStr("    ");
}
if(menu_1==2)
{
    LcdGotoXY(0,0);
晶恢复测量时的内容显示
                                LcdPrintStr("    YW Max set    ");
                                LcdGotoXY(1,0);
                                LcdPrintStr("    YW_H:");
                                LcdGotoXY(1,9);
                                // 定位到第 0 行第 7 列
                                LcdPrintNum(TD_H);
                                LcdGotoXY(1,12);
                                LcdPrintStr("%    ");
}
if(menu_1==3)
{
    LcdGotoXY(0,0);
晶恢复测量时的内容显示
                                LcdPrintStr("    GX Max set    ");
                                LcdGotoXY(1,0);
                                LcdPrintStr("    ");
                                LcdPrint_PH(1,7,PH_H);
}
}
}

```

```

}

/*****
// 报警判断
*****/

void AlarmJudge(void)
{
    uchar i;
    if(temp>Temp_H) //报警函数
    {
        {
            for(i=0;i<10;i++)
            {
                Led_P=1;
                Buzzer_P=1;
                DelayMs(50);
                Led_P=0;
                Buzzer_P=0;
                DelayMs(50);
            }
        }
    }
    if((ADC_Value[0]>TD_H)||((ADC_Value[1]>PH_H))//报警函数
    {
        for(i=0;i<10;i++)
        {
            Led_P=1;
            Buzzer_P=1;
            DelayMs(50);
            Led_P=0;
            Buzzer_P=0;
            DelayMs(50);
        }
    }
}

/*****
// 主函数
*****/

void main()
{
    LcdInit();

    Buzzer_P=0;
    // 执行液晶初始化
}

```

```

DelayMs(500);
DelayMs(500);
while(DS18B20_ReadTemp()==850)           // 等待温度传感器初始化完
成
{
    DelayMs(10);
}
while(1)
{
    KeyScanf();
    temp=DS18B20_ReadTemp();               // 获取温度传感器的温度
    LcdGotoXY(0,0);
                                           // 定位到第 0 行
    LcdPrintStr("T:");
    LcdGotoXY(0,2);
    LcdPrintTemp(temp);
    // 显示当前的温度值
    LcdGotoXY(0,7);
    LcdPrintStr("      ");
    ADC_Processing();
    LcdPrint_TD(1,0,ADC_Value[0]); // 显示
    LcdPrint_PH(1,8,ADC_Value[1]); // 显示
    AlarmJudge();
    // 判断一下是否需要报警，是的话则报警
}
}

```